EXPERIMENT-4

AIM: Write a program to implement Breadth First Search Traversal.

Tree-

```
      a
    /   \
   b     c
  / \    |
 d   e   f
```

Code-

```
% Define the tree structure
tree(a, [b]).
tree(b, [c]).
tree(c, [d]).
tree(d, [e]).
tree(e, [f]).
tree(f, []).

% BFS traversal to find the path between two nodes
bfs(Start, Goal, Path) :-
   bfs_helper([[Start]], Goal, Path).

% Base case: If the goal is found in the first path, return it
bfs_helper([[Goal|Visited]|_], Goal, [Goal|Visited]).
% Recursive case: Explore the next level (children of the current node)
bfs_helper([Path|Paths], Goal, FinalPath) :-
   extend(Path, NewPaths),
   append(Paths, NewPaths, Paths1),
   bfs_helper(Paths1, Goal, FinalPath).

% Extend the current path by adding the neighbors of the current node (level-
wise)
extend([Node|Path], NewPaths) :-
   findall([NewNode, Node|Path],
      (tree(Node, Neighbors), member(NewNode, Neighbors), \+
member(NewNode, Path)),
      NewPaths).
```

```
% Predicate to read input from the user and run BFS level-wise traversal
run_bfs :-
    write('Enter start node: '),
    read(Start),
    write('Enter goal node: '),
    read(Goal),
    (   bfs(Start, Goal, Path)
    ->  reverse(Path, ReversedPath),  % Reverse to get the correct path order
        format('BFS Path from ~w to ~w: ~w~n', [Start, Goal, ReversedPath])
    ;   format('No path found between ~w and ~w.~n', [Start, Goal])
    ).

% Example query to run the program
:- initialization(run_bfs).
```

Output-

```
?- [bfs_tree_input].
Enter start node: a
Enter goal node: f
BFS Path from a to f: [a, b, c, d, e, f]
```

```
?- [bfs_tree_input].
Enter start node: a
Enter goal node: e
BFS Path from a to e: [a, b, c, d, e]
```

```
?- [bfs_tree_input].
Enter start node: b
Enter goal node: f
No path found between b and f.
```

EXPERIMENT-5

AIM: Write a Program to implement Water Jug Problem.

CODE

```
% jug capacities (X, Y)

% state is represented as (Amount in Jug1, Amount in Jug2)

% Initial state is (0, 0), and the goal is to reach (Z, _) or (_, Z)

solve_jug_problem(X, Y, Z) :-

 bfs([(0, 0)], X, Y, Z).

% BFS for exploring possible states

bfs([(State, Path)|_], X, Y, Z) :-

goal(State, Z),

 print_solution(Path).

bfs([(State, Path)|Rest], X, Y, Z) :-

 next_state(State, X, Y, NextState),

 \+ member(NextState, Path), % avoid revisiting states

 bfs([NextState, (State, Path)], X, Y, Z).

% Goal check: If the goal is reached, print the path

goal((Z, _), Z).

goal((_, Z), Z).

% Generate the next valid state after applying an action

next_state((A, B), X, Y, (X, B)) :- % Fill Jug 1

 A < X.

next_state((A, B), X, Y, (A, Y)) :- % Fill Jug 2
```

```
 B < Y.

next_state((A, B), X, Y, (0, B)) :- % Empty Jug 1

 A > 0.

next_state((A, B), X, Y, (A, 0)) :- % Empty Jug 2

 B > 0.

next_state((A, B), X, Y, (A1, B1)) :- % Pour Jug 1 into Jug 2

 A > 0,

 B < Y,

 pour(A, B, X, Y, A1, B1).

next_state((A, B), X, Y, (A1, B1)) :- % Pour Jug 2 into Jug 1

 B > 0,

 A < X,

 pour(B, A, Y, X, B1, A1).

pour(A, B, X, Y, A1, B1) :-

 Transfer is min(A, Y - B),

 A1 is A - Transfer,

 B1 is B + Transfer.

print_solution(Path) :-

reverse(Path, RevPath),

print_steps(RevPath).

print_steps([]).

print_steps([(A, B)|Tail]) :-

 write('('), write(A), write(','), write(B), write(')'), nl,

 print_steps(Tail).
```

OUTPUT

```
[Yash@nixos:~/Repo/prolog]$ swipl jug_filling.pl
Welcome to SWI-Prolog (threaded, 64 bits, version 9.2.7)

SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.

Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

?- run.
Solutions Found:

Solution:
Step: [0,0]
Step: [4,0]
Step: [2,3]
Step: [2,0]
Step: [3,0]
Step: [3,3]
Step: [1,3]
```

```
State Possibilities for this Solution:
State [0,0]: Possible in this solution
State [4,0]: Possible in this solution
State [0,3]: Not possible in this solution
State [4,3]: Not possible in this solution
State [3,0]: Possible in this solution
State [1,3]: Possible in this solution
State [0,1]: Not possible in this solution
State [4,1]: Possible in this solution
State [2,0]: Possible in this solution
State [2,3]: Possible in this solution
State [3,3]: Possible in this solution
State [3,1]: Not possible in this solution
```

```
Solution:
Step: [0,0]
Step: [0,3]
Step: [3,0]
Step: [3,3]
Step: [4,2]
Step: [0,2]
Step: [2,0]

State Possibilities for this Solution:
State [0,0]: Possible in this solution
State [4,0]: Not possible in this solution
State [0,3]: Possible in this solution
State [4,3]: Not possible in this solution
State [3,0]: Possible in this solution
State [1,3]: Not possible in this solution
State [0,1]: Not possible in this solution
State [4,1]: Not possible in this solution
State [2,0]: Possible in this solution
State [2,3]: Not possible in this solution
State [3,3]: Possible in this solution
State [3,1]: Not possible in this solution

Solution:
Step: [0,0]
Step: [4,0]
Step: [1,3]
Step: [1,0]
Step: [0,1]
Step: [4,1]
Step: [2,3]
```